

```

#include <stdio.h>
#include <stdbool.h>

#define MAX_ROWS 100
#define MAX_COLS 100

int cols, rows = 0;

bool in_range(int new_row, int new_column)
{
    return new_row >= 0 && new_row < rows && new_column >= 0 && new_column < cols;
}

// Function to simulate one round of the game of life
void simulate_round (char grid[MAX_ROWS][MAX_COLS], char new_grid[MAX_ROWS][MAX_COLS])
{
    int directions[8][2] =
    {
        {-1, -1}, {-1, 0}, {-1, 1},
        {0, -1},      {0, 1},
        {1, -1}, {1, 0}, {1, 1}
    };

    for (int i = 0; i < rows; ++i)
    {
        for (int j = 0; j < cols; ++j)
        {
            int live_neighbors = 0;

            for (int k = 0; k < 8; ++k)
            {
                int new_row = i + directions[k][0];
                int new_col = j + directions[k][1];

                if (in_range(new_row, new_col) && grid[new_row][new_col] == '1') live_neighbors++;
            }

            if (grid[i][j] == '1')
            {
                if (live_neighbors < 2 || live_neighbors > 3) new_grid[i][j] = '0';

                else new_grid[i][j] = '1';
            }

            else
            {
                if (live_neighbors == 3) new_grid[i][j] = '1';

                else new_grid[i][j] = '0';
            }
        }
    }
}

void print_grid (char grid[MAX_ROWS][MAX_COLS])
{
    for (int i = 0; i < rows; ++i)
    {
        printf("%s\n", grid[i]);
    }
}

```

```

    }
}

int main()
{
    int rounds;
    char grid[MAX_ROWS][MAX_COLS];

    scanf("%d", &rounds);

    // Input the initial grid state
    while (scanf("%s", grid[rows]) != EOF)
    {
        rows++;
    }

    cols = strlen(grid[0]);

    // Simulate the specified number of rounds
    for (int i = 0; i < rounds; i++)
    {
        char new_grid[MAX_ROWS][MAX_COLS];

        // Perform one round of the game of life
        simulate_round(grid, new_grid);

        // Copy the updated grid to the original grid for the next round
        for (int i = 0; i < rows; ++i)
        {
            strcpy(grid[i], new_grid[i]);
        }
    }

    // Output the final state after simulation
    print_grid(grid);

    return 0;
}

```